

HUAWEI TECHNICAL TEST

Dev Documentation: Setup, System Flow & Evaluation Comparison

Integrating Backend (Express + EJS), Automation (Cron), and Data Processing (SQL)

1. Backend Development (Express.js)

Key Features & Architecture:

- **Modular Folder Structure:** Implements the `Router → Controller → Service → Repository` pattern to ensure clean, maintainable, and scalable code.
- **In-Memory Storage:** Securely manages and stores form data within an isolated server-side array inside `form.repository.js`.
- **Interactive Swagger Docs:** Swagger UI is automatically hosted and available at the `/api-docs` endpoint for seamless developer testing.
- **Global Error Handling:** Dynamically intercepts malformed JSON parsing errors to prevent server downtime and unexpected crashes.

Web UI Integration (EJS):

- **Responsive & Modern Interface:** Visually stunning presentation pages designed with a premium dark mode theme, glassmorphism aesthetics, and Google Inter typography.
- **AJAX Submission:** Forms are submitted asynchronously without refreshing the page, updating data tables on-the-fly accompanied by smooth micro-animations.

How to Run the Backend:

1. Execute `npm install` inside the backend directory to install dependencies.
2. Configure the environment variables by creating a `.env` file modeled after `.env.example`.
3. Start the server in development mode using `npm run dev`. The application will be accessible via port `3000`.

2. Automation Testing (Cron & Cleansing)

Automated Data Collection:

- **Fetch Schedule:** Scheduled to run automatically 3 times a day at `08:00`, `12:00`, and `15:00` WIB (Asia/Jakarta timezone).

- **Output Format:** Generated files are securely stored under the `/home/cron` path, adhering to the naming convention: `cron_{date}_{hours}.csv`.
- **Shell Script Protection (`collect_data.sh`):**
 - **Timeout handling:** Implements connection timeout limits (10s) and total data transfer bounds (30s).
 - **Data Cap:** Caps the maximum CSV output size to a safe threshold of 1,000 lines.
 - **Validation & Headers:** Generates accurate CSV headers and performs robust response status validation checkmarks.

Data Cleansing & Node Scheduler:

- **Automated Cleansing (`data_cleansing.sh`):** A standalone shell utility that automatically scans for `*.csv` files older than 30 days using the Linux `find` command and deletes them safely.
- **Node-Cron Service (`index.js`):** A pure JavaScript-based scheduler alternative intended for hassle-free deployment on non-Linux or serverless cloud environments.

Linux Crontab Configuration:

```
0 8,12,15 * * * /path/to/collect_data.sh
0 0 * * * /path/to/data_cleansing.sh
```

3. Data Processing (SQL Queries)

Schema & Manipulation Queries:

- **Table Schema Design (`employees`):** Converted unstructured PDF string inputs (e.g., '8 Years' or '\$150') into explicit numerical formats (`FLOAT` for Experience, `INT` for Salary) to unlock robust native database statistical calculations.
- **Query 1 (Insert Albert):** Adds Albert's record with the position 'Engineer', Join Date `2024-01-24`, Experience `2.5` years, and a Salary of `$50`.
- **Query 2 (Update Salary):** Multi-row updates to standardize all employee salaries under the 'Engineer' position to `$85`.

Analysis & Subqueries:

- **Query 3 (SUM Salary 2021 - Fixed):** Accurately aggregates total salary expenditures for all active employees during the fiscal year 2021 (Join Date `≤ 2021-12-31` and Release Date is either `NULL` or `≥ 2021-01-01`).
- **Query 4 (Top 3 Experience):** Retrieves the top 3 longest-tenured employees using `ORDER BY year_of_experience DESC LIMIT 3`.
- **Query 5 (Subquery Engineer ≤ 3 Years):** Employs an efficient subquery filtering pattern `WHERE name IN (...)` to isolate Engineers with a total tenure of 3 years or less.

4. Evaluation, Comparison & Enhancements

Key architectural adjustments and corrections introduced to close gaps from previous iterations:

1. SQL Query No. 3 Logic Correction:

- **Issue:** The previous logic strictly evaluated `join_date ≥ '2021-01-01'` , inadvertently omitting senior employees who joined prior to 2021 but remained fully active during that year.
- **Solution:** Refactored to properly evaluate contract overlap parameters (`Join_Date ≤ '2021-12-31'` and `Release_Date ≥ '2021-01-01'`).

2. Shell Script Resilience (Automation):

- **Issue:** Scripts were highly volatile and vulnerable to breaks if third-party endpoints timed out or dropped connections unexpectedly.
- **Solution:** Hardened script execution via `curl` timeout safeguards, explicit HTTP download validation steps, and an upper limit ceiling of 1,000 lines to prevent disk space exhaustion.

3. Frontend Web Backend Implementation:

- **Issue:** Complete lack of an interactive user visual endpoint and vulnerability to crash-prone unhandled JSON decoding exceptions.
- **Solution:** Successfully integrated the EJS template engine to drive a modern visual UI paired with system-wide Express middleware error trapping.